



Projeto Snake em MIPS Assembly: Otimização Arquitetural e Integração Web

Universidade Federal do Maranhão (UFMA)
Disciplina de Arquitetura de Computadores
Discentes: Alana, Arthur, Cássio e Millena



Objetivos e Visão Arquitetural

- Desenvolver a lógica de um jogo operando diretamente no Hardware.
- Gerir fisicamente a memória (Alocação, Endereços, Little-Endian).
- Controlar o fluxo do processador (Desvios e Polling).
- Construir uma ponte de comunicação assíncrona entre o simulador e um Dashboard Web.

Fundamentação Teórica: Arquitetura RISC e MIPS

- O processador MIPS utiliza a arquitetura RISC (*Reduced Instruction Set Computer*), otimizada para executar instruções simples em um único ciclo de máquina.
- Aplicação rigorosa da filosofia **Load/Store**: a Unidade Lógica e Aritmética (ULA) não processa dados diretamente da memória RAM.
- O fluxo exige que os dados sejam trazidos para os registradores (lw), calculados na ULA e posteriormente devolvidos à memória (sw).
- Essa simplicidade arquitetural permite que o jogo mantenha uma estabilidade de quadros e tempo de resposta preciso.

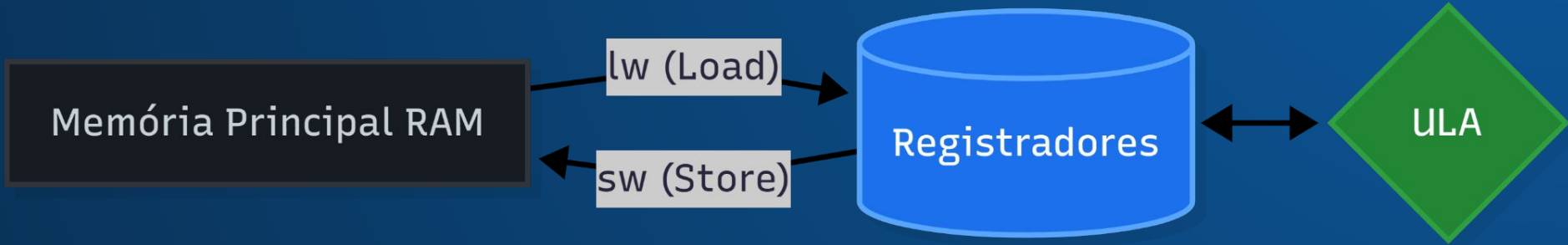
Memória Principal RAM

lw (Load)

sw (Store)

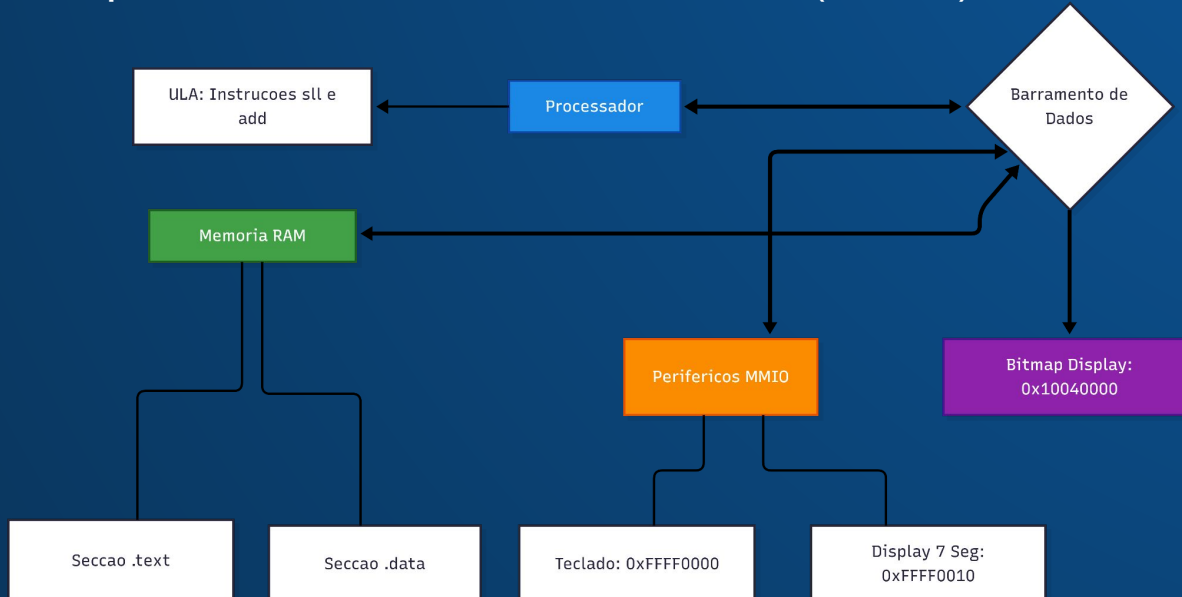
Registradores

ULA



Arquitetura do Sistema

- CPU MIPS atuando sobre o Barramento de Dados/Endereços.
- Memória Principal fragmentada em .text (Código) e .data (Variáveis).
- Periféricos mapeados diretamente na memória (MMIO).



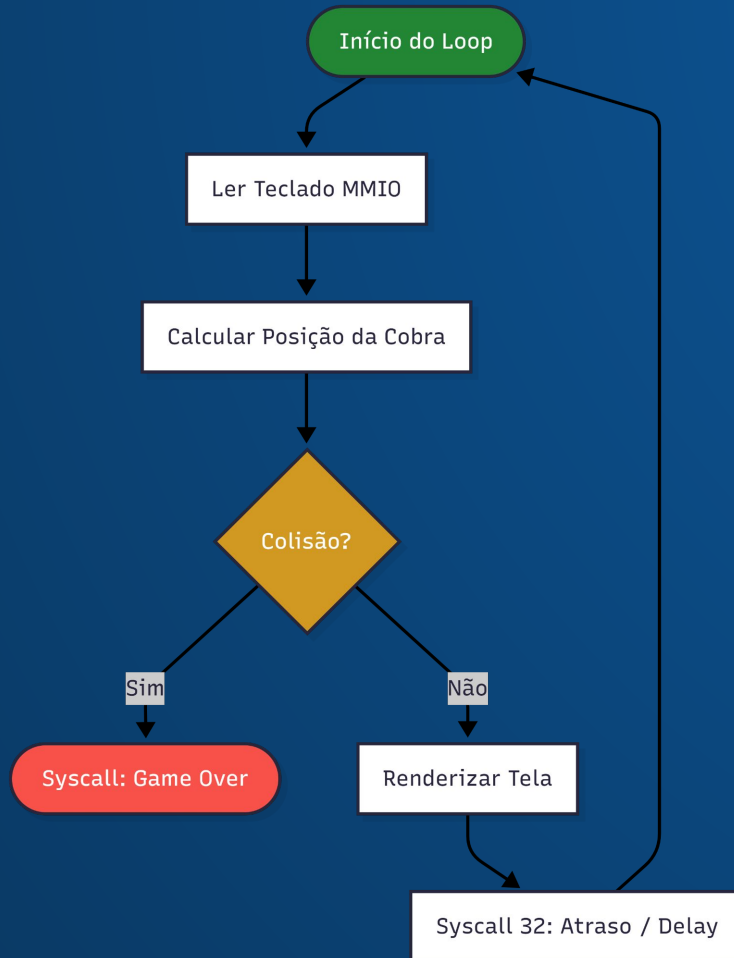
O Mapa de Memória (MMIO)

Como o processador comunica com os periféricos sem sistema operativo?

0x10040000	Tela de Vídeo (<i>Frame Buffer 64x64</i>)
0xFFFF0000	Teclado (<i>Controle / Ready Bit</i>)
0xFFFF0004	Teclado (Dados ASCII)
0xFFFF0010 / 0xFFFF0011	Displays de 7 Segmentos

Dinâmica de Execução: O Game Loop e o Ciclo de Von Neumann

- O "coração" do jogo opera por meio de um laço infinito (*Game Loop*) que simula o ciclo clássico de Busca, Decodificação e Execução.
- A cada iteração, o processador busca o status dos periféricos (teclado), decodifica as alterações de coordenadas e executa o cálculo de nova posição.
- Sem a abstração de um Sistema Operacional para gerenciar *threads*, o controle de tempo do processador (Clock) é ditado manualmente no código através de interrupções de hardware programadas (*Syscall 32*).



O Gargalo de Desempenho

$$\textit{Endereço} = \textit{Base} + [(Y * 64) + X] * 4$$

- A tela é uma matriz 2D (64x64), mas a RAM é um vetor linear.
- O Problema: A instrução nativa de multiplicação (`mult`) na ULA exige entre 10 e 32 ciclos de *clock*.
- A Consequência: Queda drástica de taxa de quadros (FPS) ao aumentar a resolução da arena de jogo.

Gestão de Memória: Estruturação do Corpo da Cobra

- O corpo da cobra não é um objeto singular, mas um conjunto de coordenadas manipuladas através de vetores paralelos (Eixo X e Eixo Y).
- A alocação dinâmica tradicional poderia causar invasão de endereços e falhas críticas de *Stack Overflow* durante o crescimento do jogador.
- **Solução de Engenharia:** Reserva estática preventiva na seção `.data`. Foi utilizada a diretiva `.space 16384` para alocar 4096 palavras contíguas (16 KB) na memória principal.
- Isso garante capacidade suficiente para a cobra preencher 100% da malha visual sem corromper as variáveis adjacentes do sistema.

Memoria_Data

+cobra_tamanho : .word 3

+velocidade : .word 400

+cobra_col : .space 16384(Vetor X)

+cobra_lin : .space 16384(Vetor Y)

lw / sw

Logica_Deslocamento

Puxa cauda para a posição anterior

Atualiza a cabeça com nova coordenada

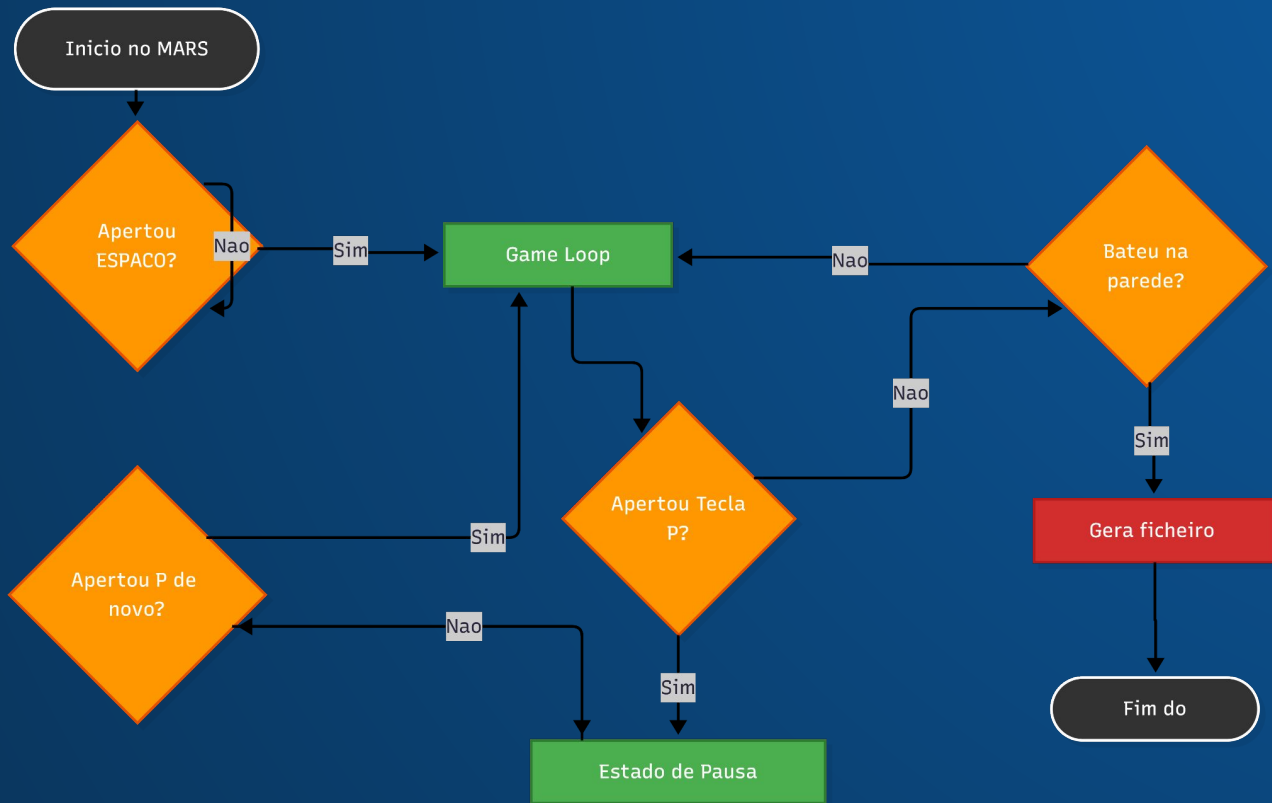
A Otimização no Nível do Bit (A Solução RISC)

```
escrever_pixel:
    # Otimização ULA: Endereço = Base + [(Y * 64) + X] * 4
    sll $t8, $a1, 6      # Desloca Y 6 bits (Multiplica Y por 64)
    add $t8, $t8, $a0    # Soma X
    sll $t8, $t8, 2      # Desloca 2 bits (Multiplica total por 4 bytes)
```

- Substituição da instrução *mult* por Deslocamento Lógico à Esquerda (*sll*).
- Como 64 e 4 são potências de base 2, o processador apenas move os bits nos registos, resolvendo a matemática em **1 ciclo de clock**.
- Resultado: Multiplicação resolvida em apenas 1 ciclo de clock. O FPS foi estabilizado.

Controle de Fluxo e Interrupção

- O MIPS executa Polling contínuo no endereço 0xFFFF0000.
- A execução apenas é travada intencionalmente, como na Tela Inicial (aguardando ESPAÇO) e no Sistema de Pausa.



O Sistema de Pausa (Congelamento do PC)

Ao detectar a tecla 'P', forçamos o Contador de Programa (*Program Counter*) a entrar num laço fechado, congelando a atualização da tela e o tempo do jogo.

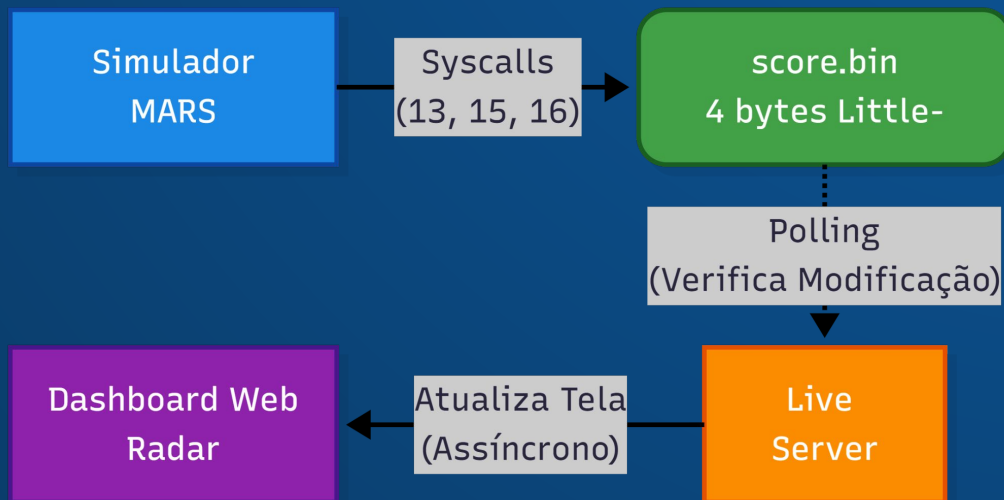
```
rotina_pausa:
    li    $t1, 0xFFFF0000
    lw    $t0, 0($t1)
    andi  $t0, $t0, 1
    beq   $t0, $zero, rotina_pausa  # Congela a execução (Laço de bloqueio)
```

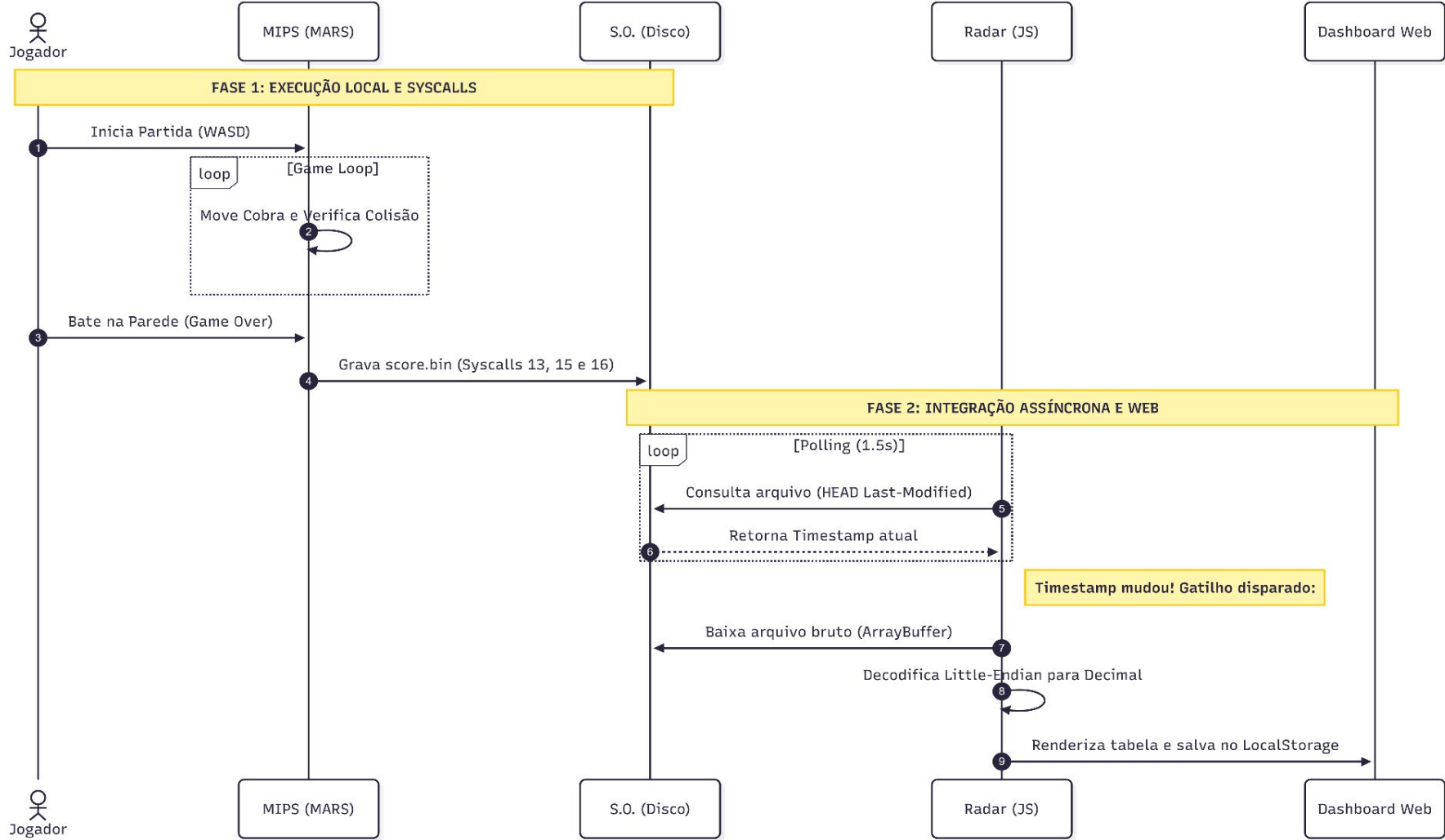
Comunicação com Hardware de Saída

- Uso da instrução de hardware `div` para separar a pontuação decimal.
- O Quociente (Dezenas) vai para o registrador L0.
- O Resto (Unidades) vai para o registrador HI.
- Conversão via *Lookup Table* e escrita física com a instrução `sb` (*Store Byte*).

O Desafio da Integração Web

- **Objetivo:** Enviar a pontuação para a nuvem de forma automática ("Zero Cliques").
- **O Obstáculo:** A *Sandbox* dos navegadores bloqueia a leitura de arquivos do disco local (*file:///*) para prevenir roubo de dados.
- Hospedar o site diretamente num servidor externo resultaria em bloqueio de acesso ao arquivo gerado pelo MARS.





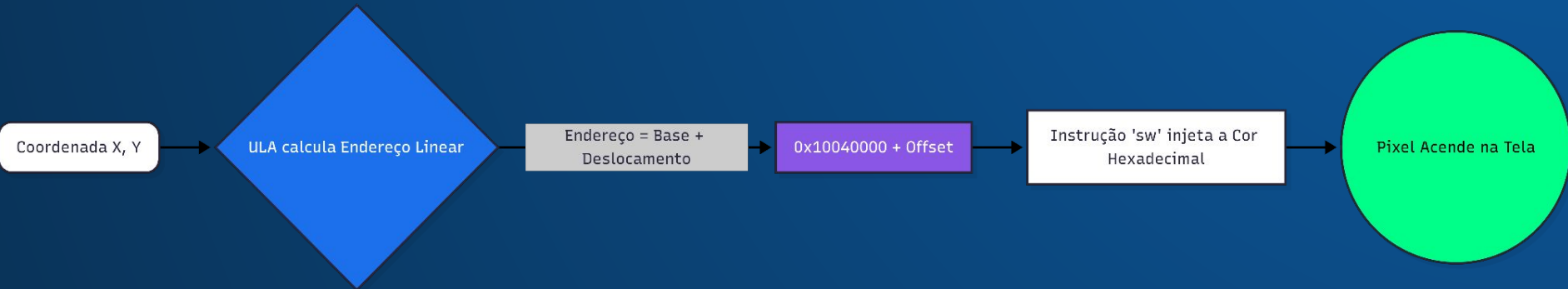
Arquitetura do Middleware

Divisão de responsabilidades para contornar a segurança de forma ética:

1. **Hardware (MARS):** Usa *Syscalls* de sistema operacional para gerar o `score.bin`.
2. **Servidor Local:** O Front-end roda em `127.0.0.1` (*Live Server*), desativando o bloqueio do navegador.
3. **Radar JavaScript:** Um algoritmo de varredura verifica assincronamente a data do arquivo e atualiza a tela instantaneamente após o *Game Over*.

Renderização Direta em Memória (Frame Buffer)

- O Bitmap Display é uma representação visual estrita de um segmento da memória RAM.
- Cada pixel na tela de 64x64 corresponde a 1 Palavra de memória (4 bytes), iniciando no endereço físico mapeado em 0x10040000.
- A alteração de cores dispensa bibliotecas gráficas. Ocorre pela injeção direta de códigos hexadecimais (ex: 0x00ff88 para a cor verde) em endereços calculados pela ULA.
- Fórmula de Mapeamento Linear: $\text{Endereço} = \text{Base} + [(Y * 64) + X] * 4$





Snake MIPS Dashboard

Integração Front-end e Ranking Global – UFMA

Passo 1: Identificação

Digite seu Nome para iniciar:

Ex: Arthur Felipe

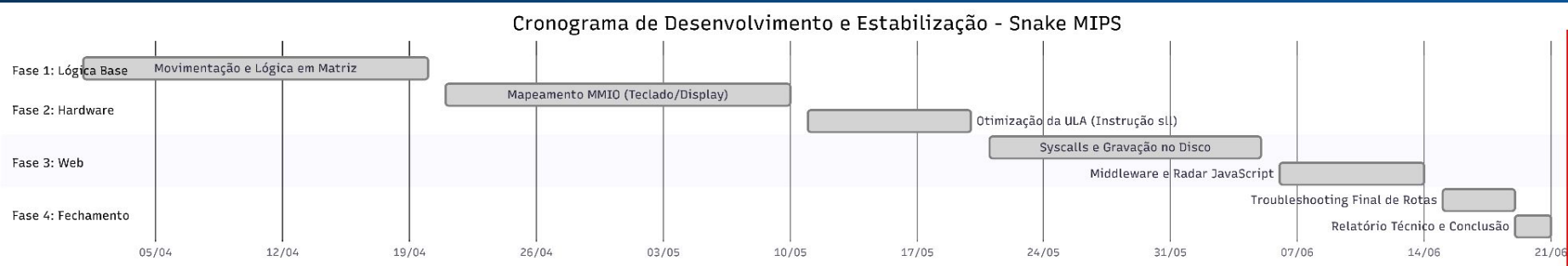
Confirmar e Ligar Radar



Tabela de Líderes

POS	JOGADOR	SCORE (BYTES)
1º	Alana	51
2º	Guilherme	43
3º	Arthur	27
4º	João	13

Gestão do Projeto: Cronograma de Execução



- **Fase 1 (Protótipo):** Validação da lógica Assembly e movimentação matricial.
- **Fase 2 (Integração):** Controle de periféricos (MMIO) e otimização de ciclos de clock.
- **Fase 3 (Expansão):** Construção da ponte entre o MIPS local e o ambiente Web.
- **Fase 4 (Estabilização):** Resolução de conflitos de hardware/software e entrega final.

Repositório e Demonstração

- Código-Fonte (GitHub):

<https://github.com/arthurmmourao/snake-assembly-mips/>

- Vídeo de Funcionamento:

https://drive.google.com/file/d/1vs9DfqczGPiMnGjPomc1mx6rPHzuoAas/view?usp=drive_link

Referências Bibliográficas

- PATTERSON, David A.; HENNESSY, John L. Computer Organization and Design: The Hardware/Software Interface. 5ª ed. Morgan Kaufmann, 2013. (Referência base para a Arquitetura RISC e conjunto de instruções MIPS).
- VOLLMAR, K.; SANDERSON, P. MARS: MIPS Assembler and Runtime Simulator. Documentação Oficial e Guia de Utilização. Disponível em: <http://courses.missouristate.edu/KenVollmar/MARS/>.
- MDN WEB DOCS. DataView - JavaScript. Mozilla Foundation. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/DataView. (Referência para a manipulação de matrizes de bytes e arquitetura Little-Endian no Frontend).
- MDN WEB DOCS. Fetch API. Mozilla Foundation. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API. (Referência para a comunicação assíncrona do Middleware Web).

Conclusão

- Sucesso na tradução da abstração de *Software* para impulsos elétricos de *Hardware*.
- Domínio rigoroso de gestão de memória e portas lógicas.
- Capacidade de resolver problemas complexos interligando o nível mais baixo da computação (Assembly) com a camada de topo (Desenvolvimento Web).